

AI 활용 기술 문서 — 냥 아레나 (Nyang Arena)

NAN 2026 사전과제 제출물 · 2026-08-05 작성 · 2026-08-10 갱신

1. 요약

이 게임에서 AI는 **게임 규칙을 만드는 자리에** 있다. 대사 생성이나 이미지 장식이 아니다.

매 판 적용되는 시너지 규칙 3개는 LLM이 생성한 풀에서 뽑힌다. 다만 **LLM 출력을 그대로 실행하지 않는다**. 출력을 좁은 DSL로 강제하고, 화이트리스트를 통과하지 못한 것은 폐기하고, 통과한 값도 게임이 감당하는 범위로 클램프한다. 전부 폐기돼도 프리셋으로 풀백해서 게임은 반드시 동작한다.

핵심 주장은 이것이다: **생성형 AI를 게임에 넣을 때 어려운 부분은 생성이 아니라 압축이다**. 모델이 무엇을 뱉든 런타임 제약이 깨지지 않도록 만드는 층이 실제 엔지니어링이고, 이 문서의 대부분은 그 층에 대한 것이다.

1.1 AI가 정한 것과 사람이 정한 것

심사자가 가장 먼저 알아야 할 경계다. **AI는 후보를 내고 코드를 쓰고 검토했다. 무엇을 채택할지는 전부 측정과 사람이 정했다**.

자리	AI가 한 것	사람이 정한 것
시너지 규칙	후보 생성	DSL의 모양, 화이트리스트, 클램프 범위, 판당 3개라는 수
음악 4곡	생성	어떤 국면에 무엇이 필요한지, 크로스페이드 0.8초, 마스터링 기준
게임 코드	대부분 작성	무엇을 만들지, 무엇을 기각할지
밸런스	스왑 실행	무엇을 찢지 정하고, 그 판정에 나를 묶었다 — 축·기준값·미달 시 exit 1까지
코드 리뷰	다른 컨텍스트에서 검토	지적을 받을지 반박할지

밸런스 행이 이 프로젝트의 성격을 가장 잘 드러낸다. 파라미터를 "이게 재밌을 것 같다"로 정한 적이 없다 — 대신 **무엇을 재야 이 게임이 게임인가**를 정했다. 결정 축 다섯과 각각의 기준값(구매 1.5 · 유물 4.0 · 지도 1.5 웨이브 ...)이 그 답이고, 관문은 미달이면 `exit 1` 을 낸다. 정한 다음에는 그 판정에 나를 묶었다 — 그래서 **마감 당일에도 관문 틀이 빨간 채로 낸다**(6.1 ③). 통과시키는 방법은 알고 있었고 쓰지 않았다.

1.2 판단 기록 셋 — 이 문서를 3분만 읽는다면 여기

① **측정된 깊이를 알고도 버린 것**. 개입은 원래 흠여짐/뭉침을 다른 키로 눌렀고, "장판 색을 읽고 판단하는" 축이 첫 보스 통과율 **+9.4%p**로 측정돼 있었다. 그것을 버리고 **버튼 하나로 합쳤다**. 1.2초짜리 예고 안에서 색을 읽는 것까지는 이 게임이 요구할 만하지만 **맞는 키를 고르는 것은 아니다**. 합친 뒤 그 축은 +0.7%p로 무너졌고, 그 수치를 표에 그대로 적었다(`submission.md` §3). 수치가 설계를 이기는 프로젝트지만, **무엇을 재는 축으로 삼을지는 수치가 못 정한다**.

② **규칙을 고치는 대신 물음을 바꾼 것**. LLM이 생성한 시너지 중 `evade_add` 계열이 반복해서 나쁜 조합을 만들었다. 개별 규칙을 고치거나 상한을 조이는 대신 **생성 축에서 그 효과를 아예 뺐다**(§4). LLM 출력을 고치는 것과 LLM에게 무엇을 물을지를 다시 정하는 것은 다른 일이고, 후자가 디렉터의 자리다.

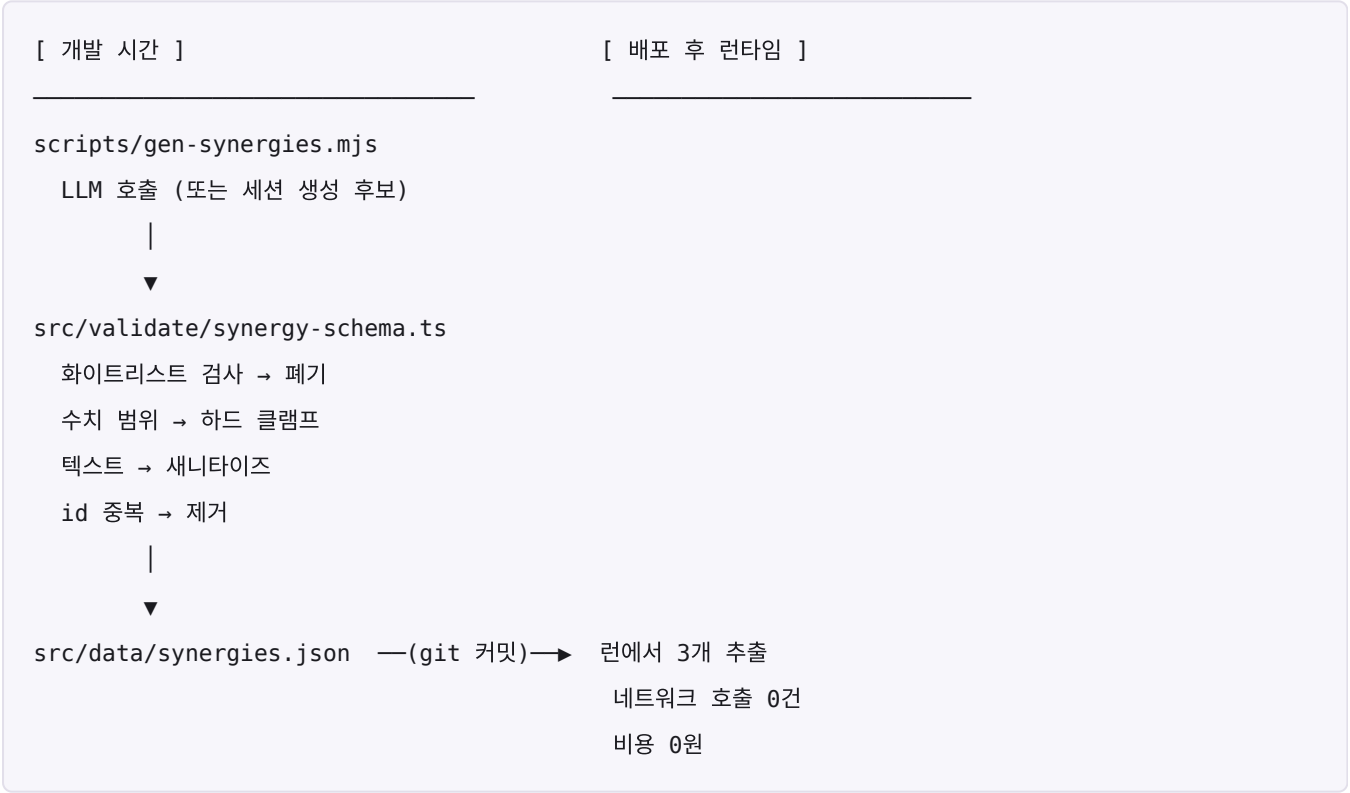
③ **통과시킬 수 있는 관문을 빨간 채로 둔 것.** 유물 측 3.3웨이브(기준 4.0). 기준을 낮추거나 유물 수치를 올리면 초록 이 된다. 후자를 실제로 시도해 1.15~1.45배를 쓸어 봤고 3.2 → 4.0 → 3.6으로 **오르내릴 뿐이라 근거가 없다고 판단 해 되돌렸다** (6.1 ③). 관문의 존재 이유가 "실행 실력이 빌드 결정을 덮으면 안 된다"인데, 이번에 덮은 것이 정확히 그 것이다.

재현 경로가 각 단계마다 붙어 있다.

생성	<code>npm run gen:synergies</code>	(키 없으면 커밋된 후보를 쓴다)
검증	<code>npm test</code>	적대 입력 11개 → 폐기 8, 통과 3(값은 놀림)
시뮬	<code>npm run sim</code>	런 300회, 시드 1~300 – 같은 시드면 같은 결과
채택	<code>npm run verify</code>	결정 측 5개를 전부 재고, 기준 미달이 있으면 <code>exit 1</code>

2. 구조

2.1 빌드타임 생성 / 런타임 소비 분리



왜 런타임 호출을 하지 않는가. 규정이 금지해서가 아니다. NAN FAQ는 "유료 API 사용료는 참가자 본인 부담을 원칙 으로 합니다"라고 명시하고, 제공 인프라 항목에서도 "유료 API 등은 참가자가 직접 준비해야 합니다"라고 한다. 금지 되는 것은 **심사자**가 별도 유료 라이선스나 승인을 요구받는 상황이지, 제출자가 자비로 프록시를 운영하는 구성이 아니 다. 즉 런타임 생성은 규정상 가능하다.

정적을 택한 이유는 **가동률**이다. 제출 링크는 심사 종료 시점까지 접근 가능해야 하는데, 사전과제는 8/10 제출 후 8/22에 결과가 나온다. 그 12일 동안 런타임 생성이 성립하려면 프록시·키·잔액·레이트리미티가 **전부** 살아 있어야 하고, 하나만 죽어도 심사자 화면에서 게임이 죽는다. 정적 생성은 그 실패 모드를 통째로 없앴다. 규정 회피가 아니라 실행 보장의 선택이다.

그리고 이 선택은 되돌릴 수 있게 설계돼 있다 — 아래 2.2를 보라.

검증 결과: 배포 번들에 XMLHttpRequest / WebSocket / 외부 URL 문자열이 0건이다. 브라우저에서 나가는 요청은 전부 동일 출처 정적 자산이다(HTML 1 + JS 1 + 스프라이트 48 + 아이콘 13 + 음악 4). Playwright로 실제 배포본을 조작하며 모든 요청의 호스트를 수집해 확인한다.

fetch(는 1개소 있다 — src/game/audio.ts 가 import.meta.env.BASE_URL 아래의 음악 파일을 받아 decodeAudioData에 넘긴다. 스프라이트를 new Image()로 받는 것과 같은 성격의, 게임이 올라간 그 자리로의 요청이다.

음악을 base64로 모듈에 박으면 fetch(문자열을 없앨 수 있었다. 하지만 그건 grep을 통과하려고 구조를 비트는 것이고, 체크 요청은 그대로 나가므로 실제로 달라지는 게 없다. 용량만 33% 붙는다. 지켜야 할 것은 "번들에 특정 문자열이 없음"이 아니라 "외부로 나가지 않음"이고, 그건 호스트를 세어서 증명한다.

반면 Vite가 기본으로 넣는 modulepreload 폴리필의 fetch(는 제거했다(modulePreload: false). 그건 실제로 발화하지 않는 코드였다 — 쓰지 않는 감사 표면을 남길 이유는 없다. 쓰는 것은 남기고 설명한다.

2.2 런타임 전환 경로

같은 스키마와 같은 검증기를 그대로 쓰면서 프록시 하나만 앞에 세우면 런타임 생성으로 승격된다. 코드 변경은 resolveSynergyPool()의 입력이 커밋된 JSON이나 응답이나 뿐이고, 검증기·클래мп·폴백은 한 줄도 바뀌지 않는다.

중요한 성질은 폴백의 방향이다. 런타임 생성이 실패하면 게임은 지금 제출한 것과 정확히 같은 상태로 떨어진다. 그래서 런타임 승격은 위험을 더하는 변경이 아니라, 잘 되면 더 얹히고 안 되면 원래대로 돌아가는 변경이다. 이 성질이 없었다면 승격을 검토하지 않았을 것이다.

3. 사용한 AI 도구

도구	용도	흔적
Claude Code (claude-opus-5)	전체 구현: 설계, TypeScript 코드, 스프라이트 슬라이싱 스크립트, 검증기, 밸런스 하네스, 문서	커밋 히스토리 전체. 각 커밋에 Co-Authored-By 명시
Claude (동일 세션)	시너지 규칙 후보 생성	scripts/synergy-candidates.json
Lyria (Google, Gemini 경유)	배경음악 4곡(준비·보스·타이틀·부검). 프롬프트는 전부 docs/audio-prompts.md에 원문 그대로	public/bgm/ 8파일. 생성 후 dB로 측정하고 마스터링한 과정은 같은 문서 §7
코드 리뷰 에이전트	구현 후 별도 컨텍스트에서 리뷰. 같은 세션에서의 self-approve를 금지하는 것이 이 저장소의 규칙이다	커밋 7b0aaf6 (17건), 커밋 1ee6743 (치명 1건)

gen-synergies.mjs는 두 입력 모드를 지원한다. OPENAI_API_KEY가 있으면 OpenAI API를 호출하고, 없으면 세션에서 생성한 후보 파일을 읽는다. 어느 쪽이든 검증기는 동일하게 적용된다. 이번 제출은 후자로 만들었다.

4. 주요 프롬프트

`scripts/gen-synergies.mjs` 의 `PROMPT` 상수 전문이다. 허용 트리거와 효과 범위는 코드의 `TRIGGERS · EFFECT_RANGE` 에서 문자열 보간으로 생성되므로, **스키마를 바꾸면 프롬프트가 자동으로 따라간다.**

너는 픽셀 고양이 오토배틀러의 시너지 규칙 디자이너다.

아래 스키마를 정확히 지키는 JSON 배열만 출력해라. 설명 문장은 쓰지 마라.

허용 trigger (이것 외에는 절대 쓰지 마라):

- same_color_3
- same_breed_2
- front_melee_2
- back_ranged_2

허용 effect.key와 값 범위 (범위를 벗어나면 잘린다):

- atk_mul: 1.1 ~ 1.6
- hp_mul: 1.1 ~ 1.6
- evade_add: 0.05 ~ 0.3
- atkspd_mul: 1.1 ~ 1.5

각 항목 형식:

```
{ "id": "소문자_스네이크", "name": "16자 이내 한글 이름",  
  "desc": "48자 이내 한 줄 설명", "trigger": "...",  
  "effect": { "key": "...", "value": 숫자 } }
```

규칙:

- trigger마다 최소 3개씩, 총 24개를 만들어라.
- id는 영문 소문자로 시작하고 중복되면 안 된다.
- 이름과 설명은 고양이답고 짧게. 수치를 설명에 쓰지 마라.

프롬프트에 범위를 적어 두지만 **지켜질 것이라고 가정하지 않는다.** 아래 검증기가 실제 계약이다.

트리거 축은 한 번 바뀌었다. 초기에는 `all_different_5` (다섯 마리를 전부 다른 품종으로)가 있었는데, 리뷰 패스가 이걸 함정으로 지적했다 — 조건을 채우려고 품종을 흘리면 강화 대상이 분산되어 팀 공격력이 31% 영구 손실되고, 되돌릴 방법이 없었다. **생성 축 자체가 나쁜 선택지를 만들어내고 있었으므로 규칙을 고치는 대신 축을 없앴다.** 대신 배치를 읽는 축 둘(`front_melee_2` , `back_ranged_2`)을 넣었다. 프롬프트는 스키마에서 보간되므로 이 변경에 자동으로 따라왔다.

5. 검증기 설계

`src/validate/synergy-schema.ts` . 빌드타임 스크립트와 런타임이 **같은 모듈**을 공유한다.

5.1 계층

계층	동작	위반 시
구조	객체인지, 필수 키가 있는지	폐기
id	<code>^[a-z][a-z0-9_]{1,31}\$</code> , 배치 내 중복 없음	폐기
trigger	TRIGGERS 화이트리스트	폐기
effect.key	EFFECT_KEYS 화이트리스트	폐기
effect.value	유한한 수	폐기
effect.value	EFFECT_RANGE 로 하드 클램프	폐기 아님 — 경계로 누름
name/desc	제어문자·꺾쇠·유니코드 서식문자 제거, 코드포인트 단위 절단	새니타이즈
name/desc	새니타이즈 후 비었는지	폐기

범위 초과를 폐기가 아니라 클램프로 처리하는 이유: 모델이 낸 아이디어(트리거 조합과 이름)는 살리고 수치만 안전하게 만드는 편이 콘텐츠 다양성에 유리하다. 반면 존재하지 않는 트리거나 효과 키는 의미를 복구할 방법이 없으므로 폐기한다.

절단을 코드포인트 단위로 하는 이유: `String.prototype.slice` 는 UTF-16 코드유닛 기준이라 이모지의 서로게이트 쌍을 반토막 낸다. 짝 잃은 서로게이트는 캔버스에 두부 글자로 렌더된다. `[...str]` 로 코드포인트 배열을 만든 뒤 자른다.

서식문자를 제거하는 이유: U+202E(RTL override)가 이름에 섞이면 `fillText` 가 뒤따르는 글자를 역순으로 그린다. XSS는 아니지만 정확히 "렌더 깨짐"이다.

5.2 적대 테스트

`npm test` — `scripts/synergy-adversarial.json` 의 규칙 위반 입력을 검증기에 통과시켜 실제로 막히는지 확인한다.

적대 입력 11개 → 폐기 8개, 통과 3개

폐기된 항목:

quad_color	허용되지 않은 trigger: same_color_4
finisher	허용되지 않은 effect.key: instant_kill
9bad_id	id 형식 위반
dup_id	id 중복: dup_id
stringy	effect.value가 유효한 수가 아님
no_effect	effect가 객체가 아님
nan_value	effect.value가 유효한 수가 아님
(문자열 항목)	객체가 아님

통과했지만 값이 늘린 항목:

dup_id	name="중복 원본" hp_mul=1.2	(중복 중 첫 번째만 생존)
tag_soup	name="태그scriptalert(1)"	(꺾쇠 제거)
overshoot	name="폭주" atk_mul=1.6	(원본 9.9 → 상한 클램프)

전부 통과 – 검증기가 적대 입력을 모두 막거나 안전한 값으로 늘렸다.

테스트는 네 가지를 단언한다: 통과분의 값이 전부 허용 범위 안일 것, 이름·설명에 꺾쇠가 없을 것, 명백히 불법인 항목이 통과하지 않을 것, 범위 초과는 폐기가 아니라 상한으로 늘릴 것.

5.3 실제 출고분

```
$ npm run gen:synergies
```

입력 모드: 세션

후보 14개 → 통과 14개, 폐기 0개 (폐기율 0.0%)

트리거별 통과 수: { same_color_3: 4, same_breed_2: 4, front_melee_2: 3, back_ranged_2: 3 }

폐기율 0%는 검증기가 늘었다는 뜻이 아니다. 출고용 후보는 스키마를 보고 만든 것이라 통과하는 게 정상이다. 검증기가 실제로 막는지는 **일부러 규칙을 어긴 입력**으로 따로 증명한다(5.2). 두 후보 파일을 분리해 둔 이유가 이것이고, 섞으면 새니타이즈된 깨진 이름이 그대로 게임에 노출된다 — 실제로 한 번 그랬다.

프롬프트는 24개를 요구하지만 출고분은 14개다. 모델이 낸 만큼만 쓰고 숫자를 맞추려고 늘리지 않았다. 한 런에서 뽑는 건 3개뿐이고 트리거당 최대 1개이므로, 측당 3~4개면 반복 감이 나지 않는다.

5.4 폴백

`resolveSynergyPool()` 은 검증 통과분이 3개 미만이면 하드코딩된 프리셋으로 채운다.

`src/data/synergies.json` 을 빈 배열로 만들어 빌드해도 게임이 정상 동작하고 프리셋 시너지(야행성 / 한배 형제 / 잡색 부대)로 플레이된다 — 실제로 확인했다.

6. AI가 만든 규칙을 사람이 아니라 시뮬레이션으로 검증한다

생성된 규칙이 밸런스를 깨는지는 눈으로 알 수 없다. 그래서 게임 로직을 브라우저 없이 그대로 돌리는 하네스를 만들었다. `scripts/balance-sim.mjs` 는 별도 구현이 아니라 `stepBattle` · `buyOffer` 를 직접 임포트하므로 **시뮬과 실제 게임이 갈라지지 않는다**.

```
$ npm run sim
런 300회 (정책: 살 수 있으면 가장 비싼 것 구매, 재배치 없음)
최소 2 p25 8 중앙값 12 p75 15 최대 24
↑ 이 값은 npm run sim이 지금 내는 것으로 갱신한다. 원본은 docs/generated/metrics-current.md
판정: 목표 구간(6~20) 안
```

이 하네스가 실제로 잡아낸 결함:

증상	원인	수정
웨이브 2에서 전멸	적은 웨이브마다 복리로 크는데 플레이어 레벨 성장은 선형	레벨 성장을 지수로
12%가 60웨이브 상한 도달, 런이 안 끝남	강화 효과는 지수인데 비용이 선형	비용도 지수로
모든 파라미터 조합이 중앙값 5로 동일	보드 만석 시 영입 오퍼가 목록에 남아 무한 재시도	불가능한 오퍼는 즉시 제거
탱키 조합이 매 웨이브 타임아웃 승리	타임아웃을 남은 체력 비율로 판정	타임아웃은 패배

마지막 항목은 생성 규칙과 직접 관련이 있다. `evade_add` 가 겹친 조합이 아무도 죽이지 못하면서 시간만 끄는 상태가 나왔고, 이는 LLM이 만든 규칙 조합이 만들어낸 상태 공간이었다. 검증기가 개별 규칙의 문법을 보장하더라도 **조합의 결과는 시뮬로만 알 수 있다**.

전투에 이동을 넣은 뒤(근접이 걸어가서 붙고 원거리는 제자리에서 쏘는 구조) 같은 하네스를 다시 돌려 도달 웨이브 분포가 유지되는지 확인했다. 전투 길이가 1~2초 늘어 타임아웃 상한을 12초에서 16초로 올린 것 외에 밸런스 파라미터는 건드리지 않았다. **게임 로직을 바꿀 때마다 눈이 아니라 이 분포를 본다**는 것이 이 프로젝트의 규칙이다.

추가로 `pickSynergies` 는 트리거당 최대 1개만 뽑는다. 같은 트리거의 규칙이 여러 개 켜지면 한 조합으로 배수가 중첩되어 밸런스가 무너지기 때문이다.

6.1 측정이 설계를 이긴 자리 — 마감 하루의 기록

이 절이 이 문서에서 가장 정직한 부분이다. **마감 당일에 조작을 크게 바꿨고, 그 결과를 측정이 세 번 뒤집었다**. 셋 다 내 직관이 틀렸고 숫자가 맞았다.

① 난이도를 어디서 되찾을 것인가 — 손잡이를 잘못 골랐다

회피가 실제로 통하도록 고치자(아래 6.2) 보스가 벽이 아니게 되어 도달 웨이브 중앙값이 12에서 25로 튀었다. 목표 구간(6~20) 밖이다. 처음엔 개입 자원을 줄이고 예고 피해를 올려 맞췄다 — 차지 6→2, 예고 피해 0.262/1.277→0.29/1.4. 중앙값은 16으로 돌아왔다. **그런데 유물 축이 무너졌다(격차 5.8 → 0.5웨이브)**.

조합	중앙값	유물 격차
차지 2 · 예고 0.29/1.4	16	0.5 ✖
차지 2 · 예고 0.262/1.277	18	-0.5 ✖
차지 2 · enemyScale 1.28	12	3.3

예고 한 대가 무거워질수록 판이 보스전 한 순간으로 수렴하고, 일반 전투력을 올리는 유물은 켈 자리를 잃는다. 전 웨이브에 걸리는 enemyScale 로 되찾자 중앙값이 12로 돌아왔을 뿐 아니라 관문 판정 **둘이 개선됐다** — 「궁합이 약하다 → 있다」, 「구매에 깊이가 없다 → 있다」.

② **도적을 약하게 만들 줄 알았는데 강해졌다** (아래 수치는 enemyScale 1.24 시절의 A/B다. 지금 값으로 다시 돌리면 절대값은 달라지고 방향만 남는다)

개선 순간 도적이 여덟 칸을 한 프레임에 건너뛰는 것을 0.3초짜리 달리기로 바꿨다. "오는 동안 얻어맞으니 도적이 약해진다"고 예상했고, 그래서 밸런스가 흔들릴 것을 각오했다. 측정은 반대였다 — **도적 물빵 평균 17.8 → 18.1, 최선인 시드 38% → 41%**. 도약은 양쪽이 하므로 적 도적도 300ms 늦게 오고, 그만큼 우리 뒷줄이 더 산다. 한쪽만 보고 세운 가설이었다.

③ **근거 없는 상향을 거부하고 관문을 빨간 채로 낸다**

유물 격차 3.3웨이브는 관문 기준 4.0에 못 미친다. 유물 보너스를 1.15~1.45배로 쏘아 봤지만 3.2 → 4.0 → 3.6 → 3.3으로 **오르내릴 뿐 단조가 아니었다** — 노이즈를 쫓고 있었다. 관문을 통과시키려고 기준을 낮추거나 근거 없이 수치를 올릴 수 있었지만 하지 않았다. 그 관문의 주석이 존재 이유를 이미 적어 두고 있다: "실행 실력이 빌드 결정을 덮으면 안 된다." 이번에 덮은 것이 정확히 그것이고, 가리는 것보다 적는 편이 낫다.

6.2 버그를 의견이 아니라 분포로 분해한다

"눌러서 피했는데 데미지가 들어온다"는 보고를 받았다. 화면을 다시 보는 대신 예고에 걸린 고양이 **113마리를 좌표로 분해**했더니 원인이 하나가 아니었다.

원인	비율	실제 결함
나갔다가 다시 들어감	40%	safeSpot 이 장판 경계 바로 바깥 을 돌려줘, 겹침 밀어내기 한 번에 도로 들어감
애초에 못 나감	38%	같은 이유 + 탐색 반경 부족
밖에 있었는데 걸어 들어감	22%	안전한 고양이는 아무 처리도 안 해서 도화선 1.2초 동안 보스로 걸어감

이 분해는 **일회 계측이다** — 저장소에 스크립트로 남기지 않았다. 6.3의 기준으로 보면 남겼어야 맞다. 대신 결론은 `npm run dodge:test` 가 계약으로 잡고 있다 ("구간 안의 고양이가 예고가 터지기 전에 빠져나온다", "구간 밖 고양이는 제자리에 붙잡힌다").

셋은 서로 다른 수정을 요구했고, 하나만 고쳤으면 "여전히 가끔 맞는다"가 됐을 것이다. 고친 뒤 **차지가 있을 때 예고에 아무도 안 맞는 비율이 39% → 100%**가 됐다. 비율로 분해하지 않았다면 첫 번째 원인만 고치고 끝냈을 것이다.

같은 계측이 설계 결함도 하나 드러냈다. 뭉치 예고는 WoW의 soak였고 **성공해도 안에 있는 고양이들이 피해를 나눠 받았다**. 규칙으로는 말이 되는데 화면에서 거짓말을 한다 — 제때 눌러 제대로 모였는데 숫자가 뜬다. 붉은 장판의 정확한 반대(안은 안 맞고 밖만 맞는다)로 바꿔 규칙을 둘에서 하나로 줄였다.

6.3 검사가 잡지 못하면 주장도 하지 않는다

새 검사를 넣을 때마다 일부러 옛 코드로 되돌려 실제로 실패하는지 확인했다. 통과만 확인하면 아무것도 안 재는 검사도 초록이다.

검사	되돌린 것	결과
개입 이동이 여러 프레임에 나뉜다	<code>startDash</code> → <code>c.fx = tx</code>	2건 실패 ✓
체력 바가 보드 안에 있는가	옛 바 공식	기기 7종 실패 ✓
도약이 여러 프레임에 나뉜다	<code>c.fx = to.fx</code>	2건 실패 ✓

기존 회피 검사는 도착 시점만 봐서 순간이동으로도 전부 통과했고, `c.alive` 로 걸러 장판 안에서 죽은 고양이를 "빠져나온 것"으로 썼다. 검사가 있다는 것과 검사가 무언가를 잡는다는 것은 다른 이야기다.

근거가 명제를 뒷받침하지 않은 사례도 하나 적는다. 체력 바를 머리 위로 올리며 "셀 안에 들어온다(probe 기기 10종 통과)"고 커밋에 적었는데, 두 군데가 틀렸다 — 그 계산은 반경 0인 일반 고양이에만 맞았고(저격수는 세로 배치 7기기에서 안내 문구를 덮었다), 근거로 든 `npm run probe` 는 고양이 기하를 아예 안 보고 있었다. 바 기하를 함수로 빼서 렌더러와 프로브가 같은 코드를 부르게 고쳤다. 이제 주장과 근거가 같은 곳을 가리킨다.

7. 그 밖의 AI 활용

7.1 검사를 사람 눈에서 스크립트로 옮기기

AI가 코드를 빠르게 쓸수록 틀렸다는 걸 알아채는 속도가 병목이 된다. 그래서 눈으로 보던 것을 전부 실행 가능한 검사로 바꿨다.

- 스프라이트 파이프라인:** 캐릭터 시트 PNG 한 장(1374×4306)에서 20종 × 6포즈를 프로그래매틱하게 잘라낸다(`scripts/slice-sprites.py`). 알고리즘 자체는 결정적이고 AI가 아니다 — AI가 한 것은 이 스크립트를 쓴 것이다. 배경 제거는 테두리에서 시작하는 flood fill로만 한다. 전역 색상 치환을 쓰면 크림색·흰색 고양이의 몸통까지 지워진다.
- 레이아웃 계약 검사:** `scripts/layout-probe.mjs` 가 기기 10종에서 두 가지를 단언한다 — (a) 준비 단계의 팀 요약 띠가 보드를 덮지 않을 것, (b) 보상 단계의 카드와 그 머리줄이 HUD·안내 문구를 덮지 않을 것. **(b)는 최근에 계약이 뒤집힌 자리다.** 카드를 세로로 세우려면 240px쯤 필요한데 그만큼 보드 예산에서 떼면 데스크톱 셀이 60px에서 30px로 반토막 난다. 그래서 카드가 보상 단계에 보드를 **의도적으로** 덮게 하고, 대신 그동안 보드가 입력을 받지 않는다는 불변식(`canRearrange()` 가 `prepare` 에서만 참)에 기대기로 했다. 프로브는 이제 "덮지 마라"가 아니라 "무엇을 덮어도 되고 무엇은 안 되는가"를 검사한다. 같은 스크립트가 카드 설명 줄바꿈 경계 10건도 함께 본다.

7.2 생성 에셋도 같은 관문을 통과시킨다 — 음악 4곡

시너지 규칙에 쓴 방법을 오디오에도 그대로 적용했다. **생성 전에 합격 조건을 먼저 적고**, 납품물을 dB로 재고, 조건을 못 채우면 편집하거나 버린다. 전문은 [docs/audio-prompts.md](#) 에 있고 여기서는 구조만 적는다.

조건 선언 → 생성 → 측정 → 마스터링 → 출고

§3

§4~6

§7

§7-5

public/bgm/

핵심 조건은 "신호음이 곡에 묻히지 않을 것"이었다. 이 게임은 붉은·청록·금빛 예고를 소리로도 알리는데, 보스곡 중역이 그 대역을 먹으면 소리를 넣은 의미가 없다. 그래서 곡을 대역별로 재고 보스곡 중역이 곡 전체보다 **16.1 dB 아래**임을 확인한 뒤에야 신호음 레벨을 정했다.

그런데 이 절에서 가장 값진 것은 측정 결과가 아니라 측정 방법을 세 번 고친 기록이다.

처음 한 것	무엇이 틀렸나	고친 것
자기상관 1등으로 BPM 판정	보스곡이 87.6 BPM으로 찍혔다. 실제로는 132.5이고 $87.6 = 132 \times 2/3$ — 셋잇단 그루브의 주기였다	후보를 셋 뺐아 요청값과 대조
대역 비중(%) 으로 판정	보스곡은 저역이 크므로 중역 비중이 낮게 나오는데, 그건 "중역이 비었다"와 다른 말이다	절대 레벨(dBFS)로
낮은 FFT 해상도	저역 구분이 뭉개졌다	해상도를 올려 재측정

셋 다 **결론이 아니라 잣대가 틀린 경우**다. 생성물을 검수할 때 위험한 것은 모델이 이상한 것을 주는 상황이 아니라, 이상한 자를 들고 "괜찮다"고 판정하는 상황이다. 6.3에 적은 것과 같은 이야기다 — 검사가 있다는 것과 검사가 무언가를 잡는다는 것은 다르다.

마지막으로 **UX 판정 하나를 소리보다 위에 뒀다**. 브라우저는 사용자 입력 전에 소리를 못 낸다. "소리를 들려주려고 시작 화면에 클릭 관문을 세울 것인가"에 아니라고 답했다 — 소리 하나 때문에 모든 플레이어에게 문턱을 세우지 않는다. 대신 첫 탭이 곧 잠금 해제가 되게 했다.

7.3 리뷰를 구현과 다른 컨텍스트에 두기

구현한 컨텍스트에서 스스로 승인하지 않는다. 방금 짠 코드를 방금 짠 머리로 보면 **의도가 코드를 대신 읽어 준다**.

첫 리뷰 패스는 17건(치명 1, 높음 4, 중간 6, 낮음 6)을 냈다. 가장 최근 패스는 카드 UI에서 치명 1건을 냈는데, 그 내용이 이 규칙의 존재 이유를 잘 보여준다.

넓은 카드에서 여백을 카드 **폭**으로 계산해 카드 **높이**에서 뺐다. 넓고 낮은 화면(가로 1136px 이상 & 세로 366px 이하)에서 초상 크기가 음수가 되고, `createRadialGradient` 가 던진 예외가 `requestAnimationFrame` 루프를 끊어 **화면이 영구히 굳는다**.

구현 컨텍스트에서는 이게 보이지 않았다. 세로 카드에서는 폭이 짧은 변이라 값이 우연히 맞았고, 그래서 "여백은 짧은 변에서 뺐아야 한다"는 사실이 코드에 드러나지 않았다. 리뷰 에이전트는 의도를 모르는 채로 `computeLayout` 을 전 뷰포트 스왑해서 조건을 줬었고, 실제 브라우저에서 재현해 콘솔 로그까지 붙여 왔다. 수정은 한 줄 (`Math.min(cr.w, cr.h)`)이었고, 방어선을 하나 더 두고 1280×300에서 재현이 사라진 것을 확인했다.

교훈은 "AI가 실수한다"가 아니다. 생성 속도가 올라가면 검토가 병목이 되고, 검토를 같은 컨텍스트에 두면 병목이 아니라 사각지대가 된다는 것이다.

8. 외부 에셋 · 오픈소스 출처

항목	출처	라이선스
고양이 스프라이트 (20종 × 6포즈)	자체 제작 (mmporong, 2026)	자체 보유
배경 · 전투 연출	자체 제작 (mmporong, 2026)	자체 보유
UI 아이콘 13장 (생선 · 직업 4 · 유물 8)	GUI Pro - Minimal Game Light (Layer Lab)	유니티 에셋스토어 EULA(유료) — 빌드 결과물에 통합해 배포하는 것은 허용, 에셋 자체를 재사용 가능한 형태로 배포하는 것은 금지
음악 4곡 (준비 · 보스 · 타이틀 · 아웃트로)	Lyria(Google, Gemini 경유)로 생성. 프롬프트 전문과 측정·마스터링 과정은 docs/audiot-prompts.md	자체 보유
TypeScript	Microsoft	Apache-2.0
Vite	Vite 팀	MIT
폰트	OS 시스템 폰트 (system-ui)	별도 임베드 없음

아이콘은 흰 실루엣을 게임 색으로 틴트해서 쓴다. 어떤 팩을 검토했고 무엇을 왜 안 썼는지는 [docs/assets.md](#) 에 있다.

런타임 의존성은 **0개**다. 게임 엔진, 상태관리 라이브러리, UI 프레임워크를 쓰지 않았다. 실측 payload는 이렇다.

	크기
JavaScript	101.1 KB (gzip 36.9 KB)
스프라이트 PNG 120장	111.7 KB
음악 OGG/MP3 8파일	1053.6 KB

한글 픽셀 폰트를 임베드하지 않는 이유는 이 표가 아니다. 예전에는 "번들 예산" 때문이라고 적었는데, 오디오 1 MB를 넣기로 한 순간 그 논변은 성립하지 않는다. 실제 이유는 다르다 — 고유 음절 270자를 5×7로 그리면 4.75 KB지만, 시너지 이름을 LLM이 생성하므로 **어떤 음절이 나올지 미리 알 수 없다**. 빠진 글자가 하나라도 있으면 그 규칙만 글자가 깨진다. 시스템 폰트는 그 위험이 없다.

타인의 저작물을 무단 사용한 부분은 없다. 유료 에셋은 위 표의 라이선스 범위 안에서만 쓴다.

9. 재현 방법

```
git clone https://github.com/mmporong/nyang-arena.git
cd nyang-arena
npm install

npm test          # 검증기 적대 테스트
npm run probe     # 기기 10종 레이아웃 겹침 검사
npm run sim       # 밸런스 시뮬레이션 300런
npm run gen:synergies # 시너지 규칙 재생성 (키 있으면 API, 없으면 세션 후보)
npm run build     # 타입체크 + 프로덕션 빌드
```